INTRODUCCIÓN A JAVA (I)

Programación

INTRODUCCIÓN

JAVA



Lenguaje de programación de propósito general y **orientado a objetos.**

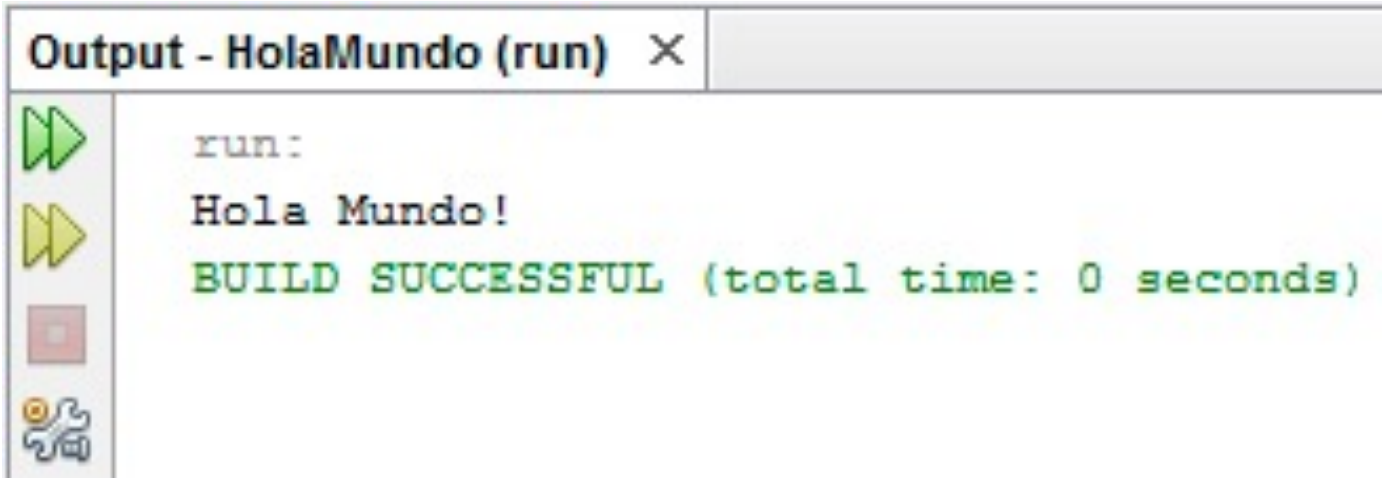
Objetivo → **el código puede escribirse una sola vez y ser ejecutado en cualquier tipo de dispositivos** (PC, móvil, etc.).

Características de Java:

- *Sencillo*
- *Orientado a Objetos*
- *Distribuido*
- *Robusto*
- *Seguro*
- *Portable*
- *Arquitectura Neutral*
- *Rendimiento medio*
- *Multithread*

ESTRUCTURA BÁSICA DE UN PROGRAMA EN JAVA

```
//  HolaMundo.java
public class HolaMundo {
        public static void main (String[] args) {
                System.out.println( "Hola Mundo" );
        }
}
```



ELEMENTOS BÁSICOS

COMENTARIOS

DE UNA LÍNEA

```
// comentarios de línea
```

DE VARIAS LÍNEAS

```
/*
        comentarios de una o
        más líneas
*/
```

DE DOCUMENTACIÓN

```
/**
        comentario de documentación, de una o más líneas
*/
```

IDENTIFICADORES

Nombran variables, funciones, clases y objetos; cualquier cosa que el programador necesite identificar o usar.

Reglas :

- **Distinción entre mayúsculas y minúsculas.**
- Formados por cualquiera de los caracteres del código Unicode. El **primer carácter no** puede ser un **dígito numérico y no** pueden utilizarse **espacios en blanco ni símbolos coincidentes con operadores**.
- La **longitud** máxima es casi **ilimitada**.
- **No** puede ser una **palabra reservada** ni los valores lógicos true o false.
- **No** pueden ser **iguales a otro identificador declarado en el mismo ámbito**.

IDENTIFICADORES

Nombran variables, funciones, clases y objetos; cualquier cosa que el programador necesite identificar o usar.

IMPORTANTE: Por convenio:

- *Los **nombres** de las **variables** y los **métodos** deberían empezar por una letra **minúscula** y los de las **clases** por **mayúscula***

- *Si el identificador está formado por **varias palabras, la primera** se escribe en **minúscula** (excepto para las clases) y el **resto** de palabras se hace **empezar por mayúscula** (por ejemplo: anyoDeCreacion).*

TIPOS DE DATOS

Java es un lenguaje **fuertemente tipado**, y esta característica es una de las razones de su seguridad y robustez.

Cada variable tiene un tipo. Cada expresión tiene un tipo. Cada tipo o clase está definida estrictamente.

DATOS SIMPLES

Permiten crear variables que almacenan un solo valor.

- Para números enteros:

  byte, **short**, **int**, **long**

- Para números reales:

  float, **double**

- Para caracteres: **char**
- Para valores lógicos: **boolean**.

DATOS OBJETO

Estructuras de datos más complejas que permiten almacenar muchos datos (vectores, objetos, etc.).

Veremos un tipo de dato compuesto llamado **String** que permite representar texto.

TIPOS DE DATOS SIMPLES

Tipo	Descripción	Memoria ocupada	Rango de valores permitidos
byte	Número entero de 1 byte	1 byte	-128 ... 127
short	Número entero corto	2 bytes	-32768 ... 32767
int	Número entero	4 bytes	-2147483648 ... 2147483647
long	Número entero largo	8 bytes	-9223372036854775808 ... 9223372036854775807
float	Número real en coma flotante de precisión simple	32 bits	$\pm 3{,}4 * 10^{-38}$ ... $\pm 3{,}4 * 10^{38}$
double	Número real en coma flotante de precisión doble	64 bits	$\pm 1{,}7 * 10^{-308}$ ... $\pm 1{,}7 * 10^{308}$
char	Un solo carácter	2 bytes	
boolean	Valor lógico	1 bit	true o false

VARIABLES Y CONSTANTES

VARIABLES

La forma básica de declarar (crear) una variable es la siguiente:

tipo identificador;

- Las variables pueden ser inicializadas en el momento de su declaración.
- Es posible declarar varias variables en una sola línea.

En resumen, la declaración de variables sigue el siguiente patrón:

tipo identificador [= valor][,identificador [= valor] ...];

CONSTANTES

Una variable precedida de la palabra **final** se convierte en una constante.

ÁMBITO

El **ámbito de una variable** es la porción del programa donde dicha variable puede utilizarse.

El ámbito de una variable depende del lugar del programa donde es declarada, pudiendo pertenecer a cuatro categorías distintas.

1. Variable local: → *Una variable local se declara dentro del cuerpo de un método de una clase y es visible únicamente dentro de dicho método.*
2. Atributo.
3. Parámetro de un método.
4. Parámetro de un tratador de excepciones.

PALABRAS CLAVE

abstract	continue	for	new	switch
boolean	default	goto	null	synchronized
break	do	if	package	this
byte	double	implements	private	threadsafe
byvalue	else	import	protected	throw
case	extends	instanceof	public	transient
catch	false	int	return	true
char	final	interface	short	try
class	finally	long	static	void
const	float	native	super	while

LITERALES

LITERALES

Son elementos que sirven para representar un valor en el código fuente del programa.

Se utilizan para establecer los valores de los tipos de datos simples (y Strings).

En Java existen literales los tipos de datos:

- Lógicos (boolean).
- Carácter (char).
- Enteros (byte, short, int y long).
- Reales (double y float).
- Cadenas de caracteres (String).

OPERADORES

OPERADORES

Son una parte indispensable de la programación ya que nos permiten realizar cálculos matemáticos y lógicos, entre otras cosas.

Los operadores pueden ser:

- **Aritméticos** : sumas, restas, etc.
- **Relacionales** : menor, menor o igual, mayor, mayor o igual, etc.
- **Lógicos** : and, or, not, etc.
- **Asignación** : =
- **Ternario**: ?:

ARITMÉTICOS

OPERACIÓN	OPERADOR	REDUCIDO	AUTO	DESCRIPCIÓN
SUMA	+	+=	++	Suma aritmética de dos operandos
RESTA	-	-=	--	Resta aritmética de dos operandos.
MULTIPLICACIÓN	*	*=		Multiplicación de dos operandos
DIVISIÓN	/	/=		División entera de dos operandos
RESTO (MÓDULO)	%	%=		Resto de la división entera (o módulo)

RELACIONALES

OPERADOR	DESCRIPCIÓN
>	1er operando mayor que 2º operando
<	1er operando menor que 2º operando
>=	1er operando mayor o igual que 2º operando
<=	1er operando menor o igual que 2º operando
==	operandos iguales
!=	operandos distintos

LÓGICOS

OPERADOR	DESCRIPCIÓN
&&	Y lógico. Devuelve true (cierto) si son ciertos op1 y op2
||	O lógico. Devuelve true (cierto) si son ciertos op1 o op2
!	Negación lógica. Devuelve true (cierto) si es false op1.

A	B	A && B	A || B	!A
True	True	True	True	False
True	False	False	True	False
False	True	False	True	True
False	False	False	False	True

DE ASIGNACIÓN

El operador de asignación es el símbolo igual: =

variable = expresión

Asigna a la variable el resultado de evaluar la expresión de la derecha.

OPERADOR TERNARIO

a	a ? b : c;
false	c
true	b

PRECEDENCIA DE OPERADORES

1. Operadores postfijos: ( ) (paréntesis)
2. Operadores unarios: ++expr, --expr, -expr, ~ !
3. Creación o conversión de tipo: new (tipo)expr
4. Multiplicación y división: *, /, %
5. Suma y resta: +, -
6. Relacionales: <, >, <=, >=
7. Igualdad y desigualdad: ==, !=
8. AND lógico: &&
9. OR lógico: ||
10. Operador ternario: ? :
11. Asignación: =, +=, -=, *=, /=, %=, ^=, &=, |=, >>=, <<=

SALIDA Y ENTRADA ESTANDÁR

SALIDA ESTÁNDAR

Se usa ***System.out*** para mostrar información por pantalla:

- ***print***(“…”) imprime texto por pantalla
- ***println***(“…”) imprime texto por pantalla e introduce un salto de línea.

Por ejemplo, para presentar el mensaje de saludo habitual por pantalla tendríamos la siguiente clase :

```
public static void main (String[] args) {
        System.out.println( "Hola Mundo" );
}
```

SALIDA ESTÁNDAR

Pueden imprimirse variables de cualquier tipo, así como combinaciones de texto y variables concatenadas con el operador +

```
public static void main (String[] args) {
        String nombre = “Pepito”;
        int edad = 25;
        System.out.println( nombre);
        System.out.println( edad );
        System.out.println( nombre + “ tiene “ + edad + “ años “);
}
```

ENTRADA ESTÁNDAR

La clase **Scanner** facilita la lectura de datos en los programas Java.

Para utilizar Scanner en el programa tendremos que hacer lo siguiente:

1. Escribir el import

La clase Scanner se encuentra en el paquete java.util por lo tanto se debe incluir al inicio del programa la instrucción:

import java.util.Scanner;

2. Crear un objeto Scanner

Tenemos que crear un objeto de la clase Scanner asociado al dispositivo de entrada. Si el dispositivo de entrada es el teclado escribiremos:

Scanner sc = new Scanner(System.in);

ENTRADA ESTÁNDAR

3. Ejemplos de lectura:

Para leer podemos usar el método nextXxx() donde Xxx indica en tipo, por ejemplo nextInt() para leer un entero, nextDouble() para leer un double, etc.

Ejemplo de lectura por teclado de un número entero:

```
int n;
System.out.print("Introduzca un número entero: ");
n = sc.nextInt();
```